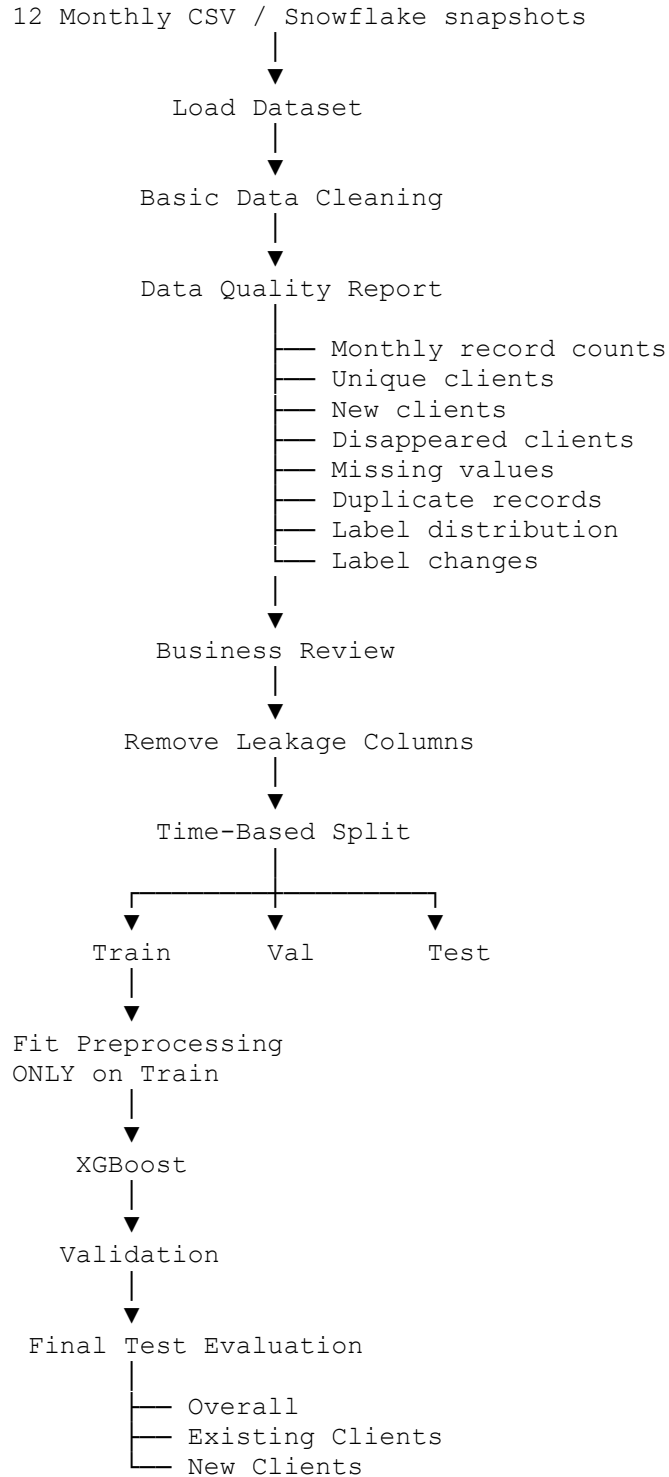


12. 所以你的完整流程应该改成这样

我建议你项目正式版本变成：



这个架构我觉得很适合你现在项目。

13. 你准备用 **GitHub Copilot**: 不要一次让它写所有东西

这是我特别建议你改的地方。

不要写:

```
Create an AML XGBoost machine learning project
using my 12 months data.
```

Copilot 很可能直接给你:

```
train_test_split(...)
```

然后什么 `leakage`、`monthly snapshot`、`new client` 都不会考虑。

应该分步骤 `prompt`。

14. 第一个 **Copilot Prompt**: 先让它分析数据，不训练

你可以直接把下面这段给 Copilot Chat:

```
I am building a multi-class AML risk scoring model using 12 months of monthly
client snapshot data.
```

```
Each monthly dataset contains largely the same clients, but:
```

- existing clients may remain across many months,
- new clients may appear,
- existing clients may disappear,
- the same `client_id` can therefore legitimately appear once in each monthly snapshot.
- The target has 3 classes: Low, Medium, and High.

```
Before performing any machine learning training, create a reusable Python
data validation and profiling module.
```

```
Requirements:
```

1. Load all monthly CSV files from data/raw/.
2. Add or preserve a snapshot_date column for each monthly dataset.
3. Combine all monthly datasets into one dataframe.
4. Validate that schemas and data types are consistent across months.
5. Report total rows and unique clients.
6. Report row count and unique client count by month.
7. Identify duplicate records based on client_id + snapshot_date.
Do NOT treat the same client appearing in different months as a duplicate.
8. Calculate new, existing, and disappeared clients between consecutive months.
9. Show target label distribution overall and by month.
10. Calculate missing-value percentages overall and by month.
11. Identify clients whose target label changed across months and calculate the percentage of clients with at least one label change.
12. Produce summary tables that can easily be exported to CSV.
13. Do not train a model yet.
14. Organize the code into clean reusable functions with type hints and docstrings.
15. Add appropriate logging rather than using excessive print statements.

Assume:

```
client identifier column = client_id
target column = risk_label
time column = snapshot_date
```

Please explain the code structure first, then generate the code file.

这个作为第一步非常合适。

15. 第二个 Prompt: Data Cleaning

等你跑完上面的 code、看过 report 后，再给 Copilot:

Based on the AML monthly snapshot profiling module we created, now create a data cleaning module.

Important:

Do NOT perform ML preprocessing such as fitting encoders, scalers, or imputers on the entire dataset.

The cleaning module should only perform transformations that are safe before train/validation/test splitting.

Requirements:

- standardize column names,
- normalize missing-value representations,
- validate required columns,
- remove only true duplicate client_id + snapshot_date records,
- validate target labels are only Low, Medium, High,

- validate snapshot_date,
- detect unexpected data types,
- identify constant columns,
- identify very-high-missingness columns,
- produce warnings rather than automatically removing questionable features,
- preserve client_id and snapshot_date for later time-based splitting.

Return the cleaned dataframe plus a data-quality summary.

这里一定告诉 Copilot:

不要自动删 questionable columns。

因为你还是要自己看。

16. 第三个 Prompt: Split Dataset

然后再让 Copilot 专门做 split:

Now create the dataset splitting module for this AML monthly snapshot dataset.

Do NOT use sklearn train_test_split or random splitting.

The dataset contains repeated client_id values across monthly snapshots. This is expected.

Use an out-of-time split:

- oldest months -> training
- following months -> validation
- most recent months -> final test

The primary evaluation should intentionally allow the same existing client to appear in earlier training months and later test months because this reflects the production use case.

However, within the final test set, create two additional populations:

1. Existing clients:
client_id appeared in the training dataset.
2. New clients:
client_id never appeared in the training dataset.

Return:
train_df
validation_df
test_df
existing_client_test_df

`new_client_test_df`

Add validation checks proving that:

- train dates occur before validation dates,
- validation dates occur before test dates,
- no future rows appear in training,
- all rows are accounted for.

Do not use `client_id` as an ML feature.

这会比只说:

`split my data`

好很多。

17. 最后才给 Copilot Training Prompt

等这些东西全部检查好了，你再让它:

Now build the final multi-class AML XGBoost training pipeline.

Use the previously created out-of-time train/validation/test split.

Important requirements:

- `client_id` must never be used as an ML feature.
- `snapshot_date` is metadata and should not be directly fed to the model unless explicitly engineered.
- preprocessing must be fitted on training data only.
- numerical features should use an appropriate imputation strategy.
- categorical features should use an appropriate imputation strategy and encoding.
- use sklearn `ColumnTransformer` and `Pipeline`.
- target classes are Low, Medium, High.
- evaluate using:
 - accuracy,
 - macro precision,
 - macro recall,
 - macro F1,
 - weighted F1,
 - per-class precision/recall/F1,
 - confusion matrix.
- pay particular attention to High Risk recall.
- evaluate separately on:
 - overall test population,
 - existing-client test population,
 - new-client test population.
- save the fitted preprocessing + model pipeline as one artifact.
- make the code production-friendly and modular.

然后如果你准备用 XGBoost:

Use XGBClassifier with a multi-class objective.

就行。

18. 我甚至建议你最终不要只有 `train.py`

你目前这个项目可以慢慢整理成:

```
project/
├── data/
│   ├── raw/
│   └── processed/
├── src/
│   ├── data_loader.py
│   ├── data_validation.py
│   ├── data_cleaning.py
│   ├── features.py
│   ├── split_data.py
│   ├── preprocessing.py
│   ├── train.py
│   └── evaluate.py
├── reports/
│   ├── data_quality/
│   ├── figures/
│   └── model_results/
├── models/
│   └── aml_risk_model.joblib
└── README.md
```

你不需要今天一次全部写出来。

但这个方向是比较好的。

19. 你现在最应该做的，其实不是 **training**

按照你目前的进度，我建议下一步顺序是:

第一步：把 12 个月数据放进去。

然后先完全不碰 XGBoost。

先回答下面这些问题：

1. 每个月多少 rows?
2. 每个月多少 unique clients?
3. 每个月多少 new clients?
4. 每个月多少 disappeared clients?
5. 有没有 client_id + month duplicate?
6. 每个月 Low / Medium / High 比例是多少?
7. 哪些 column missing 很多?
8. Missing rate 有没有某个月突然变化?
9. 哪些 column datatype 12 个月不一致?
10. 有多少 client 12 个月 label 从来不变?
11. 有多少 client Risk Rating 会改变?
12. 有没有疑似 leakage column?

这些全部看明白以后，再决定最后 features，再训练。

这会让 Copilot 一上来直接帮你写一个 500 行的 train.py 好很多。

而且从你学习的角度，我也建议你让 Copilot 一次只生成一个 **module**，然后你自己逐行看懂。尤其是 split_data.py、preprocessing.py 和 train.py，这三个文件你后面最好做到每一行都知道为什么存在。

你现在给 GitHub Copilot 的 preprocessing prompt 可以这样改

I am working with 12 months of Fenargo AML monthly snapshot data for a 3-class risk model: Low, Medium, and High.

A major limitation of the historical dataset is that not all features used by the Fenargo risk engine are stored in the EOM table. In particular, some Auto-High rule attributes are missing.

Therefore, multiple rows may have exactly the same observable model feature values but different risk labels.

Do NOT automatically remove, deduplicate, relabel, or majority-vote these conflicting observations.

Create a diagnostic module that:

1. Identifies identical observable feature combinations that have multiple target labels.
2. Calculates:
 - number of unique feature patterns,
 - number of conflicting feature patterns,
 - number and percentage of rows affected by conflicting patterns.
3. For every conflicting pattern, reports the count and percentage of Low, Medium, and High labels.
4. Classifies patterns into:
 - deterministic: only one label,
 - mostly consistent: dominant label $\geq 80\%$,
 - highly conflicting: dominant label $< 80\%$.
5. Reports which target classes are most represented among conflicting observations.
6. Produces CSV summary outputs for further review.
7. Do not modify the training dataset yet.
8. Keep client_id and snapshot_date as metadata, not model features.
9. Structure the code into reusable Python functions with type hints and docstrings.

这一步跑完之后，我建议你先看 **conflict rate**，再决定训练策略。

